**OBJECT ORIENTED PROGRAMMING**
**CCS20704, DCS20504, CCS23803**

---

**Received Date : 3 September 2026**
**Submission Due Date: 23 October 2026**
**Lecturer : Dr. Jamal Abdullahi**
**Weightage : 15%**
**Semester : SEPTEMBER 2026**

---

**Instruction to students:**

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

| **Student declaration:** | |
|---|---|
| ***I declare that:***<br>• ***This assignment is my/our own work.***<br>• ***I/we understand what is meant by plagiarism.***<br>• ***My lecturer has the right to deduct my marks in case of:***<br>- ***Late submission***<br>- ***Any plagiarism found in my assignment.*** | |
| **NAME: KAREEM SALEH WAHBY**<br>**MATRICS NO: 012026020633**<br>  | **NAME: TAN GUANYE**<br>**MATRICS NO:012026020497**<br>  |
| | |

## Introduction

Online shopping systems allow customers to browse products, add items to a cart, and place orders conveniently. This project demonstrates the concepts of Object-Oriented Programming (OOP), especially Encapsulation, by creating classes such as Product, Customer, and Order.

---

## Class 1: Product

**Product Class** represents the products available in the online shopping system. It stores important information such as product ID, name, category, price, stock quantity, brand, and rating. This class uses encapsulation by keeping all attributes private and providing public methods to access and modify them. It also includes a method to display product information.

```
/**
 * Stores a product's identifying details, price, stock, brand, and
rating and
 * can display key product information.
 */
public class Product {

    private int productId;
    private String productName;
    private String category;
    private double price;
    private int stockQuantity;
    private String brand;
    private double rating;

    /** Creates a product with default field values. */
    public Product() {
    }

    /** Creates a product and initializes all product details. */
    public Product(int productId, String productName, String
category,
                   double price, int stockQuantity,
                   String brand, double rating) {

        this.productId = productId;
        this.productName = productName;
        this.category = category;
        this.price = price;
```

```
        this.stockQuantity = stockQuantity;
        this.brand = brand;
        this.rating = rating;
    }

    /** Returns this product's identifier. */
    public int getProductId() {
        return productId;
    }

    /** Updates this product's identifier. */
    public void setProductId(int productId) {
        this.productId = productId;
    }

    /** Returns this product's name. */
    public String getProductName() {
        return productName;
    }

    /** Updates this product's name. */
    public void setProductName(String productName) {
        this.productName = productName;
    }

    /** Returns the category this product belongs to. */
    public String getCategory() {
        return category;
    }

    /** Updates the category this product belongs to. */
    public void setCategory(String category) {
        this.category = category;
    }

    /** Returns this product's unit price. */
    public double getPrice() {
        return price;
    }

    /** Updates this product's unit price. */
    public void setPrice(double price) {
        this.price = price;
    }

    /** Returns the number of units currently in stock. */
    public int getStockQuantity() {
```

```
        return stockQuantity;
    }

    /** Updates the number of units currently in stock. */
    public void setStockQuantity(int stockQuantity) {
        this.stockQuantity = stockQuantity;
    }

    /** Returns the brand of this product. */
    public String getBrand() {
        return brand;
    }

    /** Updates the brand of this product. */
    public void setBrand(String brand) {
        this.brand = brand;
    }

    /** Returns this product's rating. */
    public double getRating() {
        return rating;
    }

    /** Updates this product's rating. */
    public void setRating(double rating) {
        this.rating = rating;
    }

    /** Prints this product's name, price, and available stock. */
    public void displayProduct() {
        System.out.println("Product: " + productName);
        System.out.println("Price: RM" + price);
        System.out.println("Stock: " + stockQuantity);
    }
}
```

## Class 2: Customer

**Customer Class** is designed to store customer information within the online shopping system. It contains details such as customer ID, name, email, phone number, address, membership level, and wallet balance. This class allows the system to manage customer data securely using constructors, getters, and setters.

```
/**
 * Stores a customer's contact, membership, and wallet information
and can
 * display the customer's details.
 */
public class Customer {

    private int customerId;
    private String customerName;
    private String email;
    private String phone;
    private String address;
    private String membershipLevel;
    private double walletBalance;

    /** Creates a customer with default field values. */
    public Customer() {
    }

    /**
     * Creates a customer and initializes all customer details.
     */
    public Customer(int customerId, String customerName,
                    String email, String phone,
                    String address, String membershipLevel,
                    double walletBalance) {

        this.customerId = customerId;
        this.customerName = customerName;
        this.email = email;
        this.phone = phone;
        this.address = address;
        this.membershipLevel = membershipLevel;
        this.walletBalance = walletBalance;
    }

    /** Returns this customer's identifier. */
    public int getCustomerId() {
        return customerId;
    }
```

```
    /** Updates this customer's identifier. */
    public void setCustomerId(int customerId) {
        this.customerId = customerId;
    }

    /** Returns this customer's name. */
    public String getCustomerName() {
        return customerName;
    }

    /** Updates this customer's name. */
    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    /** Prints the customer's contact, membership, and wallet
details. */
    public void displayCustomer() {
        System.out.println("Customer: " + customerName);
        System.out.println("Email: " + email);
        System.out.println("Phone: " + phone);
        System.out.println("Address: " + address);
        System.out.println("Membership Level: " + membershipLevel);
        System.out.println("Wallet Balance: RM" + walletBalance);
    }
}
```

## Class 3: Order

**Order Class** manages order-related information and operations. It stores details such as order ID, date, quantity, total amount, payment method, order status, and shipping address. Additionally, this class includes methods to calculate the total bill and display order details, making it essential for processing customer purchases.

```
public class Order {
    private int orderId;
    private String orderDate;
    private int quantity;
    private double totalAmount;
    private String paymentMethod;
    private String orderStatus;
    private String shippingAddress;
    public Order() {
    }
    public Order(int orderId, String orderDate,
                 int quantity, double totalAmount,
                 String paymentMethod,
                 String orderStatus,
                 String shippingAddress) {
        this.orderId = orderId;
        this.orderDate = orderDate;
        this.quantity = quantity;
        this.totalAmount = totalAmount;
        this.paymentMethod = paymentMethod;
        this.orderStatus = orderStatus;
        this.shippingAddress = shippingAddress;
    }
    public double calculateBill(double price, int quantity) {
        return price * quantity;
    }
    public void displayOrder() {
        System.out.println("Order ID: " + orderId);
        System.out.println("Total Amount: RM" + totalAmount);
    }
}
```

## Class 4:ONLINE SHOPPING SYSTEM

This is the **Main Class** that serves as the entry point of the program. It demonstrates how objects of different classes interact with each other in the online shopping system. The class creates instances of Product, Customer, and Order, then performs various operations such as displaying information, calculating bills, processing payment options, and generating order summaries.

```
import java.util.Scanner;

/**
 * Runs a console-based shopping example, from product selection
through payment
 * and order summary. made by kareem saleh
 */
public class OnlineShoppingSystem {

    /**
     * Creates the sample customer, product, and order, then handles
the
     * shopping interaction with the user.
     */
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Set up the sample product and customer used in this
shopping session.
        Product product =
                new Product(101,
                        "Laptop",
                        "Electronics",
                        3500,
                        20,
                        "Dell",
                        4.8);

        Customer customer =
                new Customer(1,
                        "Kareem",
                        "kareem@gmail.com",
                        "0123456789",
                        "Selangor",
                        "Gold",
                        5000);

        Order order =
```

```
                new Order();

        // Show the customer and product details before asking for a
purchase.
        customer.displayCustomer();
        product.displayProduct();

        // Restrict the requested quantity to the product's
available stock.
        int quantity = readIntInRange(
                sc, "Enter Quantity (1-" +
product.getStockQuantity() + "): ",
                1, product.getStockQuantity());

        // Calculate and display the cost of the selected items.
        double total =
                order.calculateBill(product.getPrice(), quantity);

        System.out.println("Total Bill = RM" + total);

        // Apply free shipping when the purchase total exceeds RM
2,000.
        if(total > 2000) {
            System.out.println("You get free shipping!");
        } else {
            System.out.println("Shipping fee applies.");
        }

        System.out.println("\nPayment Methods");
        System.out.println("1. Cash");
        System.out.println("2. Credit Card");

        // Accept only one of the two listed payment choices.
        int choice = readIntInRange(sc, "Choose payment method (1-
2): ", 1, 2);
        // Convert the numeric choice into a readable payment method
name.
        String paymentMethod = choice == 1 ? "Cash" : "Credit Card";
        System.out.println(paymentMethod + " Selected");

        // Summarize the product, quantity, total, and chosen
payment method.
        System.out.println("\nOrder Summary");
        System.out.println("Product: " + product.getProductName());
        System.out.println("Quantity: " + quantity);
        System.out.println("Total: RM" + total);
        System.out.println("Payment Method: " + paymentMethod);
```

```
        // Print one numbered line for every item in the order.
        for(int i = 1; i <= quantity; i++) {
            System.out.println("Item " + i);
        }

        // Release the input resource after the shopping interaction
is complete.
        sc.close();
    }

    /**
     * Reads an integer from the user and repeats the prompt until
it falls
     * within the inclusive minimum-to-maximum range.
     *
     * @param scanner input source used to read the user's response
     * @param prompt message shown before each input attempt
     * @param minimum smallest accepted integer
     * @param maximum largest accepted integer
     * @return the valid integer entered by the user
     * @throws IllegalStateException if the input source has no more
lines
     */
    private static int readIntInRange(
            Scanner scanner, String prompt, int minimum, int
maximum) {
        // Keep requesting input until a valid in-range integer is
entered.
        while (true) {
            System.out.print(prompt);
            // Stop with an explicit error if the user input has
ended.
            if (!scanner.hasNextLine()) {
                throw new IllegalStateException("No input was
provided.");
            }

            String input = scanner.nextLine().trim();
            int value;
            // Convert the text to an integer and handle non-numeric
input.
            try {
                value = Integer.parseInt(input);
            } catch (NumberFormatException exception) {
                System.out.println("Please enter a whole number.");
```

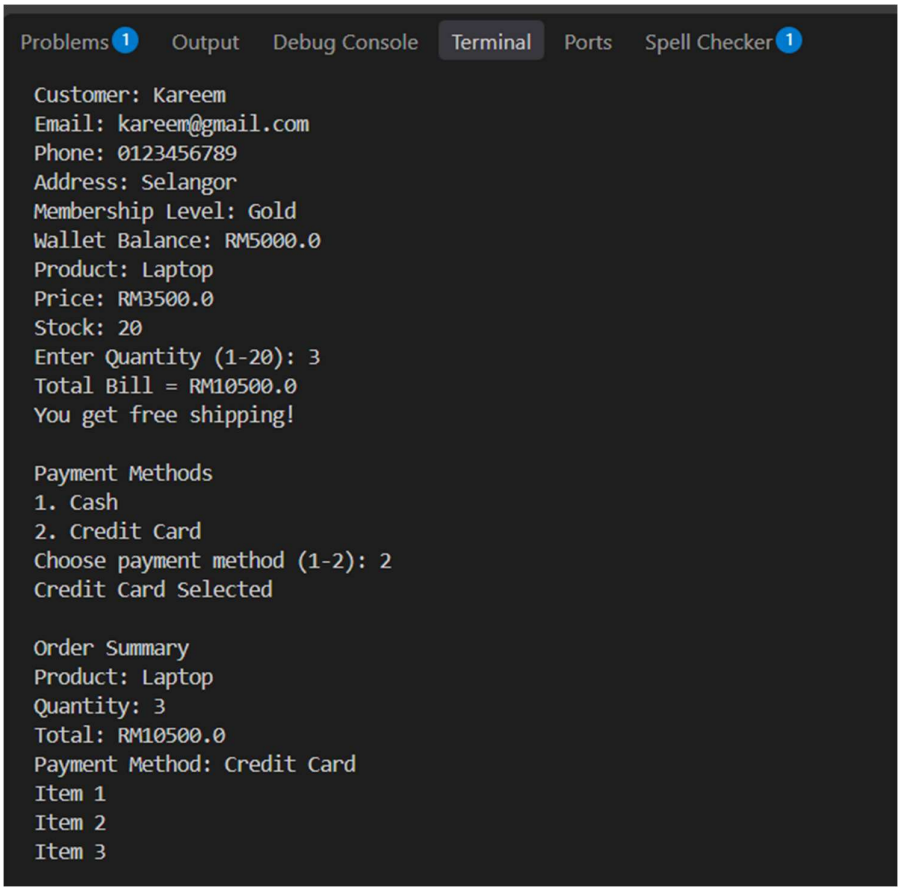
```
                // Skip range checking and ask for another input
line.
                continue;
            }

            // Return the value only when it is inside the inclusive
range.
            if (value >= minimum && value <= maximum) {
                return value;
            }

            // Explain the allowed range before the loop prompts
again.
            System.out.println("Please enter a number from "
                    + minimum + " to " + maximum + ".");
        }
    }
}
```

---

**Output**



## Conclusion

In conclusion, this Online Shopping System demonstrates the implementation of Object-Oriented Programming concepts in Java. The project successfully applies encapsulation through multiple classes, constructors, getters, setters, and methods. Furthermore, control structures such as if statements, switch statements, and loops are used to enhance program functionality and user interaction. This project provides a simple yet effective simulation of a real-world online shopping application.

## RUBRIC for Assignment One

**CLO1 – Describe the concepts of object oriented programming. (C5, PLO1)**

| COMPONENT | CRITERIA / WEIGHTAGE (%) | LEVELS of ACHIEVEMENTS/ DESCRIPTORS | | | | | |
|---|---|---|---|---|---|---|---|
| | | Excellent (4) | Good (3) | Average (2) | Needs Improvement (1) | Not Attempted (0) | Marks |
| **CONTENT (20%)** | Object Design Understanding (5%) | Clear and accurate explanation of class, attributes, methods, and constructors using correct OOP terms | Mostly correct explanation with minor gaps | Basic explanation but lacks clarity | Limited or incorrect explanation | No explanation | |
| | Encapsulation Concept (5%) | Strong understanding; correct use of private attributes and getters/setters | Good understanding with minor issues | Basic understanding but inconsistent | Weak understanding | No attempt | |
| | OOP Principles Explanation (5%) | Clearly explains class, object, and methods with examples | Mostly clear explanation | Basic explanation | Poor explanation | Not explained | |
| | Presentation Knowledge & Understanding (5%) | Confident explanation using OOP terms; answers questions clearly | Good understanding | Basic explanation | Weak understanding | No participation | |
| **PROCESS (40%)** | Class Implementation (10%) | Fully meets requirements (≥7 attributes, 2 constructors, getters/setters, ≥5 methods) | Mostly complete with minor issues | Partially complete | Incomplete | Not attempted | |
| | Encapsulation Implementation (10%) | Majority attributes private; proper getters/setters used | Mostly correct | Some inconsistencies | Poor implementation | Not implemented | |
| | Control Structures in Main Class (10%) | Effective use of if/switch and loops integrated with object | Correct with minor issues | Basic use | Minimal/incorrect use | Not used | |
| | Code Logic & Interaction (10%) + (Presentation) | Clear logic; meaningful interaction using methods | Mostly logical | Basic logic | Weak logic | Not functional | |
| **PRODUCT (40%)** | Functionality & Output (5%) | Program runs correctly with accurate output | Mostly correct output | Basic output with issues | Incorrect output | Not working | |
| | Creativity & Object Design (5%) | Highly creative, real-world, advanced methods | Creative and relevant | Basic design | Generic design | Not appropriate | |
| | Code Readability & Comments (5%) | Clean, well-organized, fully commented code | Mostly readable | Limited comments | Poor readability | No comments | |
| | Report Quality & Documentation (10%) | Well-structured report; clear explanation; proper formatting | Mostly organized | Acceptable | Poor documentation | Not submitted | |
| | Presentation & product Knowledge (15%) | Confident explanation using OOP terms; answers questions clearly | Good understanding | Basic explanation | Weak understanding | No participation | |
| | | | | | | **TOTAL** | |